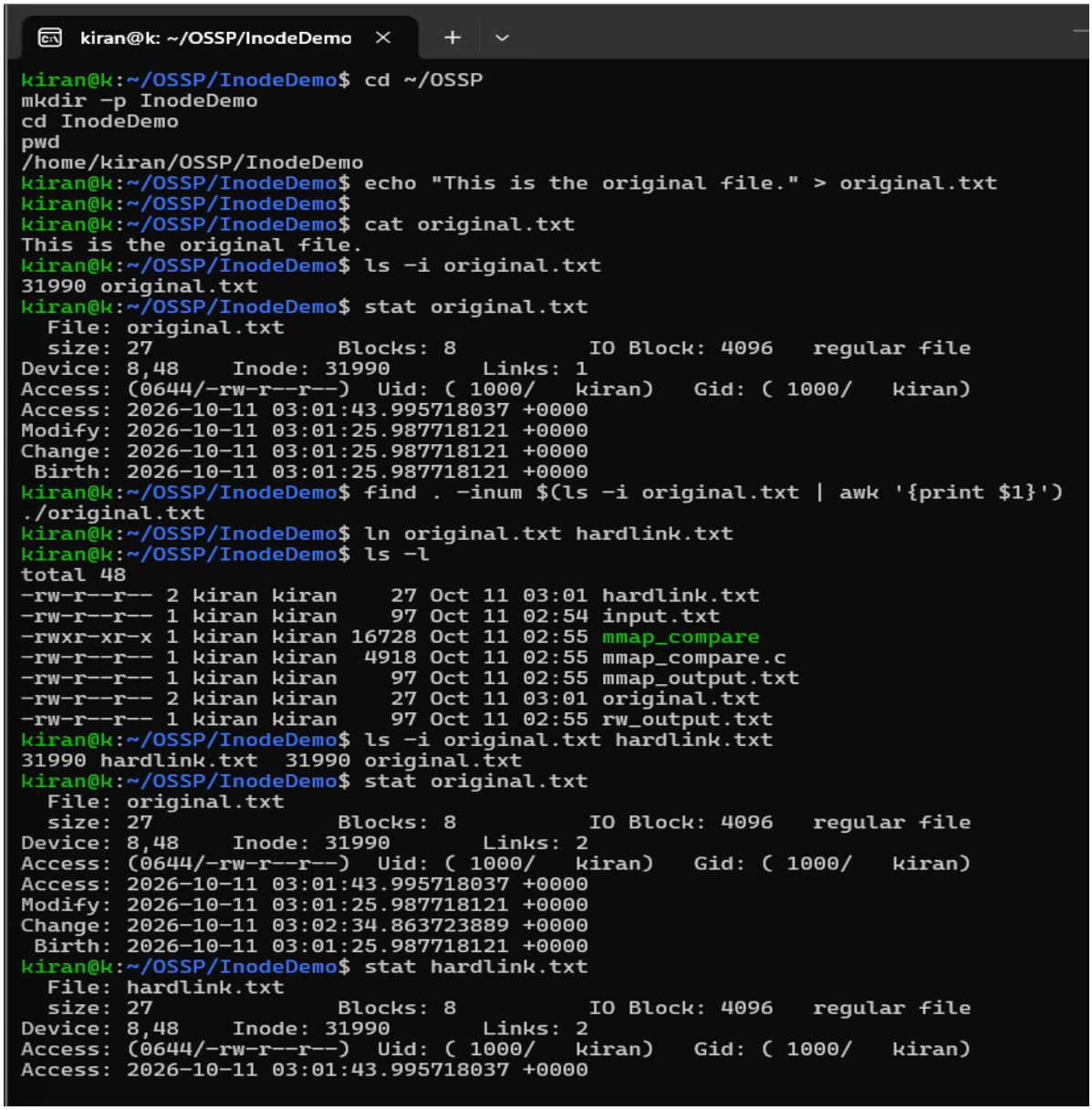
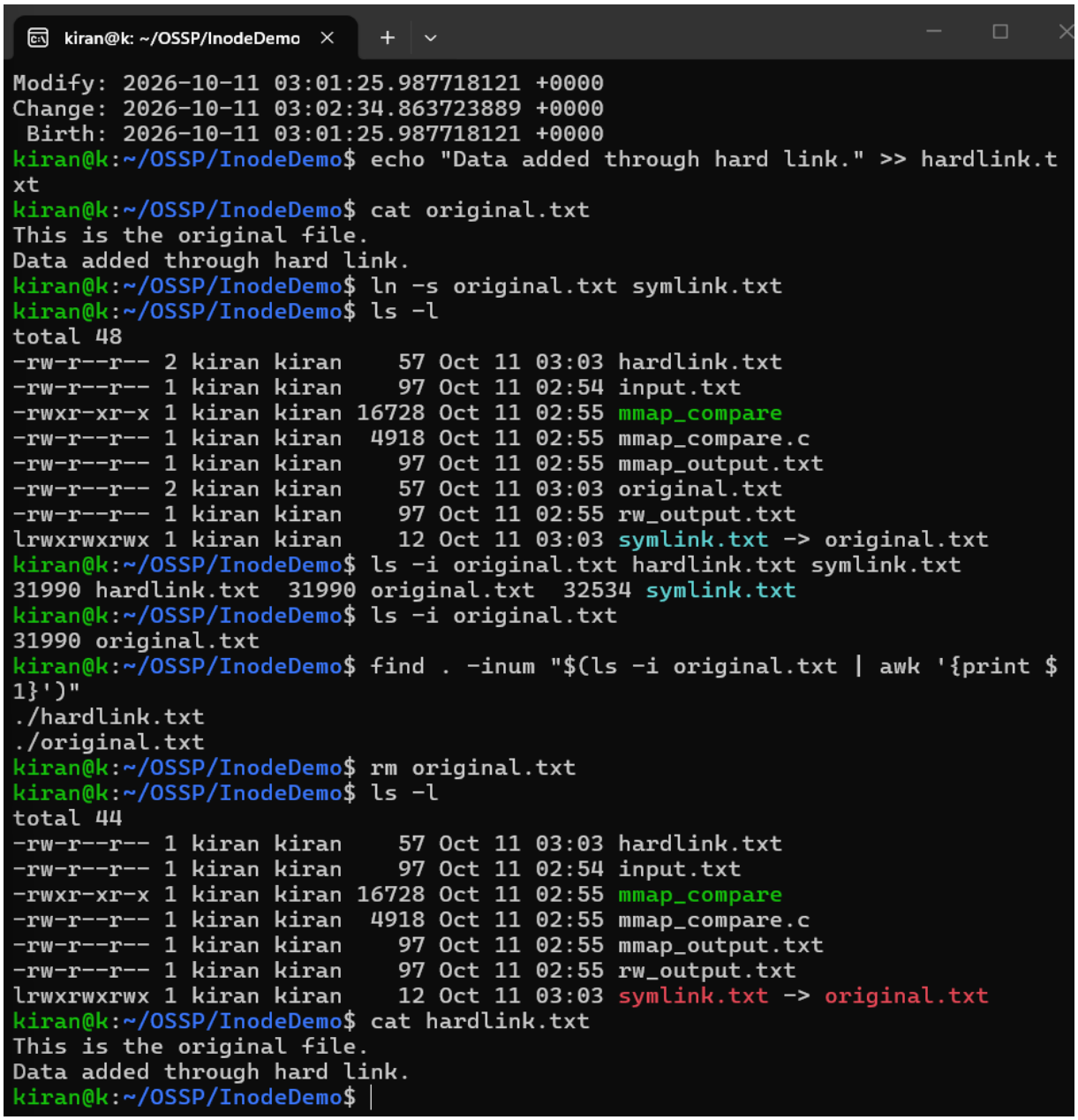
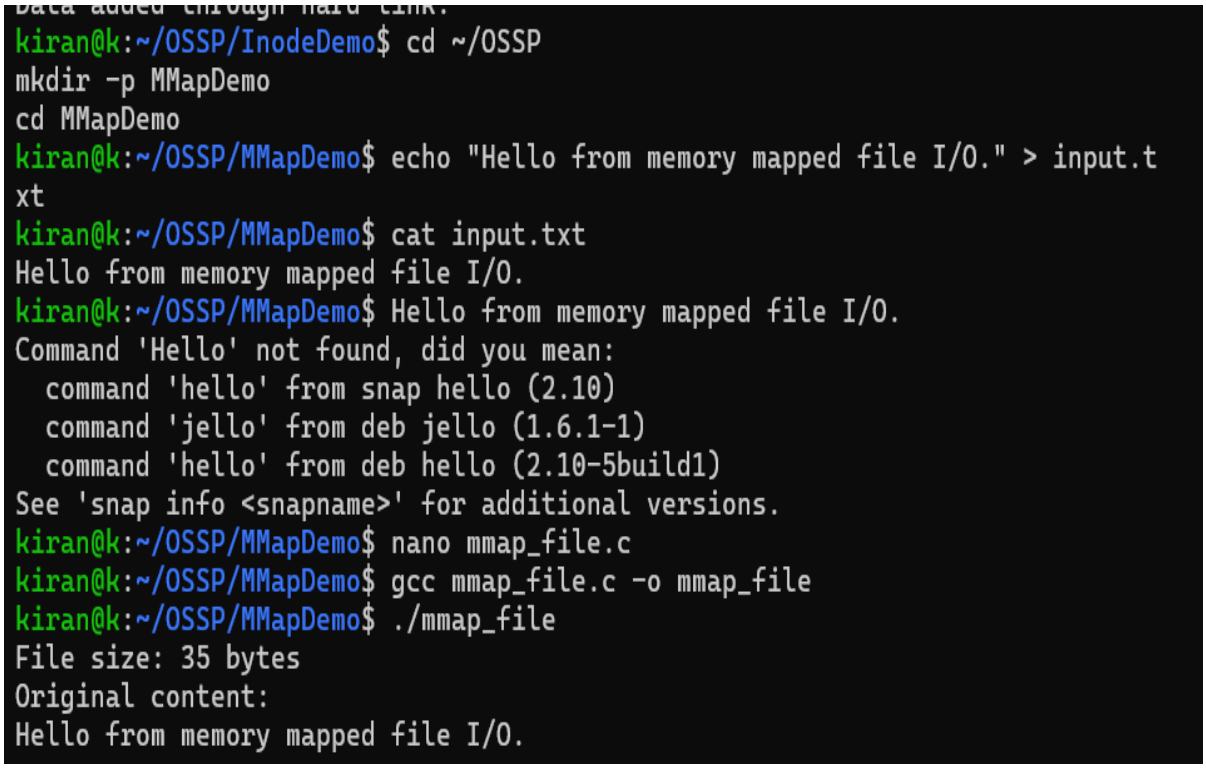
Practical Week-10

Task 1: Investigate inode structures using commands such as ls -i, stat, and find. Create hard links and symbolic links and explain their impact on inode allocation**.**




kiran@k: ~/OSSP/InodeDemo
Modify: 2026-10-11 03:01:25.987718121 +0000
Change: 2026-10-11 03:02:34.863723889 +0000
Birth: 2026-10-11 03:01:25.987718121 +0000
kiran@k:~/OSSP/InodeDemo$ echo "Data added through hard link." >> hardlink.t
xt
kiran@k:~/OSSP/InodeDemo$ cat original.txt
This is the original file.
Data added through hard link.
kiran@k:~/OSSP/InodeDemo$ ln -s original.txt symlink.txt
kiran@k:~/OSSP/InodeDemo$ ls -l
total 48
-rw-r--r-- 2 kiran kiran 57 Oct 11 03:03 hardlink.txt
-rw-r--r-- 1 kiran kiran 97 Oct 11 02:54 input.txt
-rwxr-xr-x 1 kiran kiran 16728 Oct 11 02:55 mmap_compare
-rw-r--r-- 1 kiran kiran 4918 Oct 11 02:55 mmap_compare.c
-rw-r--r-- 1 kiran kiran 97 Oct 11 02:55 mmap_output.txt
-rw-r--r-- 2 kiran kiran 57 Oct 11 03:03 original.txt
-rw-r--r-- 1 kiran kiran 97 Oct 11 02:55 rw_output.txt
lrwxrwxrwx 1 kiran kiran 12 Oct 11 03:03 symlink.txt -> original.txt
kiran@k:~/OSSP/InodeDemo$ ls -i original.txt hardlink.txt symlink.txt
31990 hardlink.txt 31990 original.txt 32534 symlink.txt
kiran@k:~/OSSP/InodeDemo$ ls -i original.txt
31990 original.txt
kiran@k:~/OSSP/InodeDemo$ find . -inum "$(ls -i original.txt | awk '{print $
1}')"
./hardlink.txt
./original.txt
kiran@k:~/OSSP/InodeDemo$ rm original.txt
kiran@k:~/OSSP/InodeDemo$ ls -l
total 44
-rw-r--r-- 1 kiran kiran 57 Oct 11 03:03 hardlink.txt
-rw-r--r-- 1 kiran kiran 97 Oct 11 02:54 input.txt
-rwxr-xr-x 1 kiran kiran 16728 Oct 11 02:55 mmap_compare
-rw-r--r-- 1 kiran kiran 4918 Oct 11 02:55 mmap_compare.c
-rw-r--r-- 1 kiran kiran 97 Oct 11 02:55 mmap_output.txt
-rw-r--r-- 1 kiran kiran 97 Oct 11 02:55 rw_output.txt
lrwxrwxrwx 1 kiran kiran 12 Oct 11 03:03 symlink.txt -> original.txt
kiran@k:~/OSSP/InodeDemo$ cat hardlink.txt
This is the original file.
Data added through hard link.
kiran@k:~/OSSP/InodeDemo$

Task 2: Design a program using mmap() to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.



```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>

int main()
{
    int fd;
    struct stat file_info;
    char *mapped;

    // Open the file
    fd = open("input.txt", O_RDWR);

    if (fd < 0)
    {
        perror("open");
        return 1;
    }

    // Get file size
    if (fstat(fd, &file_info) < 0)
    {
        perror("fstat");
        close(fd);
        return 1;
    }

    printf("File size: %ld bytes\n", (long)file_info.st_size);

    // Map file into memory
    mapped = mmap(NULL,
                  file_info.st_size,
                  PROT_READ | PROT_WRITE,
                  MAP_SHARED,
                  fd,
                  0);

    if (mapped == MAP_FAILED)
    {
        perror("mmap");
        close(fd);
        return 1;
    }

    // Read data through memory
    printf("Original content:\n%s\n", mapped);

    // Modify file through mapped memory
```

```
    // Modify file through mapped memory
    const char *new_text = "Modified using mmap file I/O.\n";

    if (strlen(new_text) <= (size_t)file_info.st_size)
    {
        memcpy(mapped, new_text, strlen(new_text));

        // Synchronize changes with file
        if (msync(mapped, file_info.st_size, MS_SYNC) < 0)
        {
            perror("msync");
        }
    }
    else
    {
        printf("New text is larger than the existing file.\n");
    }

    // Unmap memory
    munmap(mapped, file_info.st_size);

    close(fd);

    printf("File modified successfully using mmap().\n");

    return 0;
}
```

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;
    int output_fd;
    char buffer[1024];
    ssize_t bytesRead;

    fd = open("input.txt", O_RDONLY);

    if (fd < 0)
    {
        perror("open input");
        return 1;
    }

    output_fd = open("output_readwrite.txt",
                     O_WRONLY | O_CREAT | O_TRUNC,
                     0644);

    if (output_fd < 0)
    {
        perror("open output");
        close(fd);
        return 1;
    }

    while ((bytesRead = read(fd, buffer, sizeof(buffer))) > 0)
    {
        write(output_fd, buffer, bytesRead);
    }

    if (bytesRead < 0)
    {
        perror("read");
    }

    close(fd);
    close(output_fd);

    printf("File copied using read() and write().\n");

    return 0;
}
```